DEEP LEARNING LAB REPORT

Name: Ram Kumar Singh

Roll No: 2501940002

Course: MCA (AI & ML)

Final Report: Deep Learning Experiments (FNN, CNN, LSTM)

---

1. Introduction

Objective of the Project

The objective of this project is to understand and implement different deep learning architectures for various types of data:

- Tabular data → Feedforward Neural Network (FNN)
- Image data → Convolutional Neural Network (CNN)
- Sequential data → Long Short-Term Memory (LSTM)

The goal is to learn preprocessing, model building, training, evaluation, and analysis of performance.

Brief Explanation of Each Experiment

Experiment 1: Feedforward Neural Network (FNN)

A basic neural network used for classification on structured/tabular data. It consists of fully connected layers.

Experiment 2: Convolutional Neural Network (CNN)

Used for image classification. It extracts spatial features using convolution and pooling layers.

Experiment 3: LSTM for Sequence Modeling

Used for time-series prediction. It captures temporal dependencies using memory cells.

---

2. Methodology

Dataset Description

FNN (Income Prediction Dataset)

- Features: Age, education, occupation, etc.

- Target: Income (>50K or ≤50K)

CNN (Fashion-MNIST Dataset)

- 60,000 training images, 10,000 test images
- 10 clothing categories
- Image size: 28×28 grayscale

LSTM (Stock Price Dataset)

- Time-series data (closing price)
- Used to predict future values

Preprocessing Steps

FNN:

- Missing value handling
- Label encoding of categorical features
- Feature scaling using StandardScaler
- Train-test split

CNN:

- Pixel normalization (0–255 → 0–1)
- Reshaping to include channel dimension
- Train-validation split

LSTM:

- Feature selection (Close price)
- MinMax normalization
- Sequence windowing (e.g., 60 time steps)
- Train-test split

Model Architectures

FNN Architecture:

- Input Layer
- Hidden Layer 1 (ReLU)

- Hidden Layer 2 (ReLU)
- Output Layer (Sigmoid)

CNN Architecture:

- Conv2D + ReLU
- MaxPooling
- Conv2D + ReLU
- MaxPooling
- Flatten
- Dense Layer
- Output Layer (Softmax)

LSTM Architecture:

- LSTM Layer (return_sequences=True)
- LSTM Layer
- Dense Output Layer

---

3. Implementation Details

Framework Used •
TensorFlow / Keras

Hyperparameters

Parameter	FNN	CNN	LSTM
Optimizer	Adam	Adam	Adam
Loss Function	Binary CE	Categorical CE	MSE

Batch Size	32	32	32
Epochs	20	10	10
Learning Rate	Default (Adam)		

---

4. Results

Performance Metrics

Model	Accuracy	Loss	Other Metrics
FNN	~85%	Low	Validation Accuracy
CNN	~90%	Low	Precision, Recall, F1
LSTM	RMSE Low	MSE Low	Prediction Curve

Graphs (Included in Submission)

- Training vs Validation Accuracy
- Training vs Validation Loss
- Confusion Matrix (CNN)
- Prediction vs Actual Graph (LSTM)

---

5. Discussion

Observations

FNN:

- Works well for structured data
- Limited ability for complex patterns

CNN:

- Excellent performance on images
- Learns spatial features effectively

LSTM:

- Captures time dependencies
- Better than traditional models for sequence data

Challenges Faced

- Handling missing values in dataset
- Choosing correct hyperparameters
- Avoiding overfitting
- Understanding sequence modeling

Comparison of Models

Feature	FNN	CNN	LSTM
Data Type	Tabular	Image	Sequential
Strength	Simple	Feature extraction	Time memory
Complexity	Low	Medium	High

---

6. Conclusion

Key Learnings

- Different models are suited for different data types
- Preprocessing is crucial for performance
- Overfitting can be controlled using techniques like Dropout
- Deep learning models can achieve high accuracy when properly tuned

---

Real-World Applications

- FNN → Banking, customer prediction
- CNN → Face recognition, medical imaging
- LSTM → Stock prediction, weather forecasting, NLP

---

7. References

- TensorFlow Documentation
- Keras Documentation
- UCI Machine Learning Repository
- Fashion-MNIST Dataset
- Yahoo Finance (for stock data)